

AI-Powered Learner and Placement Intelligence Platform

Project Documentation

Team Members

Amit Kumar Mandal

Akshat Walvekar

Harsh Pateriya

Ansh Khare

June 1, 2026

Contents

1	AI-Powered Learner and Placement Intelligence Platform	2
2	Architectural Overview and How It Works	2
2.1	Frontend Client Application	2
2.2	Java Spring Boot Backend Service	2
2.3	Python FastAPI Machine Learning Service	2
2.4	Node.js Real-Time Notification Service	2
2.5	MySQL Relational Database	2
3	Tech Stacks Used	3
3.1	Frontend Architecture	3
3.2	Backend Core	3
3.3	Machine Learning Microservice	3
3.4	Real-Time Notification Microservice	4
3.5	Database Engine	4
3.6	DevOps, CI/CD, and Hosting Infrastructure	4
4	Codebase and Workspace Structure	4
5	Local Setup and Startup Instructions	5
5.1	1. Database Setup	5
5.2	2. Node.js Notification Service Setup	5
5.3	3. Python FastAPI Machine Learning Service Setup	5
5.4	4. Java Spring Boot Backend Setup	6
5.5	5. React Frontend Setup	6
6	Containerized Setup Using Docker Compose	6
7	Continuous Integration and Delivery (CI/CD)	7
7.1	Infrastructure Target Parameters	7
7.2	Pipeline Deployment Flow	7
8	REST API Documentation and Testing	8
9	Route Maps and Page Access Configurations	8

AI-Powered Learner and Placement Intelligence Platform

The AI-Powered Learner and Placement Intelligence Platform is an enterprise-grade intelligence system designed to track, manage, and predict learner performance and placement readiness. By combining standard administrative workflows, real-time feedback logging from mentors, predictive machine learning models, and real-time socket-based notification feeds, the platform provides placement offices and training institutions with actionable insights to optimize placement outcomes.

Architectural Overview and How It Works

The platform operates as a distributed microservices system consisting of five core components. The interaction flow between these components is detailed below.

Frontend Client Application

A highly responsive, dark-themed dashboard application built using React and Vite. It serves three distinct user roles: Administrators, Mentors, and Learners. The frontend communicates with the Java Spring Boot Backend via REST APIs using Axios, and maintains a persistent bi-directional connection with the Node.js Notification Service using Socket.IO Client.

Java Spring Boot Backend Service

The central orchestration layer of the system. It handles all security configurations, user authentication, role-based authorization, CRUD operations on the core database models, and transactional operations. It acts as the gateway to the database and integrates with both the Python Machine Learning service (for generating predictions) and the Node.js Notification Service (for publishing system events).

Python FastAPI Machine Learning Service

A microservice dedicated to calculating and predicting placement readiness. It loads a trained Random Forest model to calculate a percentage readiness score and classifies learners into categories (Placement Ready, Moderate, or High Risk) based on three primary skill scores (coding, aptitude, and communication). If the model file is not yet trained, the service gracefully falls back to a rule-based classification algorithm.

Node.js Real-Time Notification Service

An asynchronous server that manages real-time event broadcasting. It maintains active WebSocket connections with active frontend clients, mapping their specific roles and user IDs to socket connections. The Spring Boot backend triggers notifications by sending a REST POST request to this service, which immediately maps the target user and emits a real-time notification packet to the corresponding frontend client.

MySQL Relational Database

The central database engine that stores persistent records. It holds all schemas related to users, learners, mentors, assessments, feedbacks, and AI predictions. The Java backend

uses Spring Data JPA and Hibernate to interact with the database and automatically generate or update the schema.

Tech Stacks Used

The application leverages a robust stack of modern frameworks, libraries, and DevOps tools.

Frontend Architecture

- **React 19** — Component-driven UI rendering
- **Vite** — Fast development tooling and production bundling
- **React Router DOM 7** — Dynamic client-side routing and protected route guards
- **Axios** — Promise-based HTTP client for secure REST API communication
- **Recharts** — Interactive SVG charting library for dashboard analytics
- **Socket.IO Client** — Real-time WebSocket client connection management
- **CSS3** — Custom variables, dark theme variables, and grid layouts

Backend Core

- **Java 21** — Modern LTS programming language
- **Spring Boot 4.0.6** — High-performance backend application framework
- **Spring Security** — Robust access control and route protection
- **Spring Data JPA** — Object-relational mapping and database repository pattern
- **Hibernate** — Underlying database persistence provider
- **MySQL Connector J** — High-performance driver for MySQL database connectivity
- **ModelMapper** — Object-to-object mapping for converting entities to DTOs
- **JSON Web Token (JWT)** — Secure generation and parsing of stateless JWT authentication credentials
- **Springdoc OpenAPI / Swagger UI** — Auto-generation of interactive REST API documentation

Machine Learning Microservice

- **Python 3.10.11** — Primary programming language for data analysis and ML models
- **FastAPI** — Ultra-lightweight, high-performance web framework for Python APIs
- **Uvicorn** — ASGI web server for hosting high-performance FastAPI applications
- **Scikit-Learn** — Random Forest model training and inference pipelines
- **Pandas** — High-performance data manipulation and analysis library

- **NumPy** — Vectorized numerical computations
- **Joblib** — Lightweight model serialization and persistence

Real-Time Notification Microservice

- **Node.js 18** — Asynchronous server runtime environment
- **Express** — Minimalist routing and server framework for Node.js
- **Socket.IO** — Event-driven bi-directional communication library
- **CORS** — Express middleware for configuring cross-origin resource access control

Database Engine

- **MySQL 8.0** — Enterprise relational database server

DevOps, CI/CD, and Hosting Infrastructure

- **Docker** — Containerization of services for environment parity
- **Docker Compose** — Orchestration of multiple multi-container Docker applications
- **Jenkins** — Declarative pipeline automation for continuous integration and delivery
- **AWS EC2** — Hosting infrastructure for running Docker-orchestrated services
- **AWS S3** — Highly durable object storage hosting static React frontend builds

Codebase and Workspace Structure

The project workspace is structured as follows:

- `backend/` — Contains the Spring Boot Maven project
- `learner-intelligence-frontend/` — Contains the React + Vite frontend application
- `notification-service/` — Contains the Node.js Socket.IO server
- `data/` — Contains CSV datasets for training the machine learning model
- `models/` — Contains serialized Python joblib files for the Random Forest model
- `main.py` — FastAPI entrypoint script for ML inference
- `data_manager.py` — Python utility script to generate mock CSVs and train the Random Forest model
- `requirements.txt` — Dependencies for the Python machine learning service
- `Dockerfile.python` — Docker container specification for the Python FastAPI service
- `docker-compose.yml` — Orchestration script for the entire platform container stack
- `Jenkinsfile` — Declarative pipeline definition for automated builds and deployment

Local Setup and Startup Instructions

To run the application locally without Docker containers, execute the setup processes for each service in the following order.

1. Database Setup

1. Ensure MySQL Server is running locally on port 3306.
2. Log in to your MySQL terminal or database client using the username `your_username` and password `your_password`.
3. Execute the SQL command to create the required database: `CREATE DATABASE learner_db;`
4. The Spring Boot backend will auto-generate all required database tables when it is booted up.

2. Node.js Notification Service Setup

1. Navigate to the notification-service subdirectory:
`cd notification-service`
2. Install all required Node packages using npm:
`npm install`
3. Run the server locally on port 3001:
`node server.js`
4. The service will report that it is running on `http://localhost:3001`.

3. Python FastAPI Machine Learning Service Setup

1. Navigate to the root directory where the Python scripts reside.
2. Create a Python virtual environment (recommended):
`python -m venv venv`
3. Activate the virtual environment:
 - On Windows Command Prompt: `venv\Scripts\activate.bat`
 - On Windows PowerShell: `venv\Scripts\Activate.ps1`
 - On macOS/Linux: `source venv/bin/activate`
4. Install all required Python packages listed in `requirements.txt`:
`pip install -r requirements.txt`
5. Generate the initial mock CSV datasets and train the Random Forest classification model:
`python data_manager.py`
This generates mock CSVs inside the `data/` folder and outputs the serialized model `placement_rf.pkl` inside the `models/` folder.

6. Start the FastAPI application server using Uvicorn on port 8000:
`uvicorn main:app --host 0.0.0.0 --port 8000`
7. The ML API will now be listening for prediction requests on `http://localhost:8000`.

4. Java Spring Boot Backend Setup

1. Navigate to the backend directory:
`cd backend/learner-placement-backend`
2. Compile, resolve dependencies, and package the Spring Boot application using Maven:
`mvn clean package -DskipTests`
3. Run the built jar file:
`java -jar target/learner-placement-0.0.1-SNAPSHOT.jar`
4. Alternatively, import the Maven project directly into your preferred IDE (Eclipse/IntelliJ) and run the `LearnerPlacementBackendApplication.java` class.
5. The server will boot up and be accessible on `http://localhost:8081`.

5. React Frontend Setup

1. Navigate to the frontend directory:
`cd learner-intelligence-frontend`
2. Install the required dependencies:
`npm install`
3. Start the local Vite development server:
`npm run dev`
4. The application will boot up on `http://localhost:5173`. Open this address in your web browser to log in and interact with the dashboards.

Containerized Setup Using Docker Compose

If you have Docker and Docker Compose installed, you can launch the entire infrastructure stack (MySQL, Python ML API, Node.js Socket.IO, and Java Spring Boot Backend) using a single orchestrator command.

1. Ensure ports 3306, 8000, 3001, and 8081 are not in use on your host machine.
2. Open your terminal at the root directory of the repository.
3. Rebuild the service containers from scratch to ensure clean builds:
`docker compose build --no-cache`
4. Launch the entire stack in detached background mode:
`docker compose up -d`
5. All database schemas are automatically generated inside the containerized MySQL, and the services communicate seamlessly using Host Networking Mode.

6. To shut down the environment, run:

```
docker compose down
```

Continuous Integration and Delivery (CI/CD)

The project includes a declarative Jenkins pipeline configured in the `Jenkinsfile`. The pipeline automates the validation, building, and deployment process.

Infrastructure Target Parameters

- **AWS Default Region:** `ap-southeast-2`
- **Target AWS S3 Bucket Name:** `ai-powered-learninig-platform` (Static website hosting)
- **Target AWS EC2 Host IP Address:** `YOUR_EC2_IP` (Docker Compose runtime container host)

Pipeline Deployment Flow

Stage 1: Checkout

Clones the latest code changes from the master/main version control repository.

Stage 2: Build & Deploy Backend (EC2 via Docker Compose)

Runs directly on the Jenkins agent environment. It shuts down the existing container fleet, rebuilds the containers without local caching to inject changes, and launches the container stack:

- `docker compose down`
- `docker compose build --no-cache`
- `docker compose up -d`

Stage 3: Build & Deploy Frontend (S3)

Enters the `learner-intelligence-frontend` directory. It programmatically injects production environment variables pointing to the target EC2 deployment IP:

- `VITE_API_URL=http://YOUR_EC2_IP:8081`
- `VITE_SOCKET_URL=http://YOUR_EC2_IP:3001`

It then installs dependency trees and executes the production-optimized compiler:

- `npm install`
- `npm run build`

Finally, it syncs the compiled static asset distribution package in the `dist/` folder to the AWS S3 bucket, removing stale remote assets:

- `aws s3 sync dist/ s3://ai-powered-learninig-platform --delete`

REST API Documentation and Testing

The Java Spring Boot backend includes an auto-generated Swagger UI interface, allowing developers to inspect endpoints, review schema requests, and perform manual testing.

- While the backend is running locally on port 8081, open your web browser and navigate to:
`http://localhost:8081/swagger-ui/index.html`
- You can interactively test administrative endpoints, learner records, assessment logging, and prediction APIs.
- Endpoints marked with `@PreAuthorize` in the backend source code require a JWT Bearer Token in the HTTP Authorization headers. To test these endpoints, first log in through the Auth controller, copy the returned token, and paste it into the Swagger Authorization input field.

Route Maps and Page Access Configurations

The React Frontend application implements strict, client-side route guards for role-based view controls. The available routes and access restrictions are listed below.

Route	Access
/	Login view, open to all users
/admin	Admin Dashboard, restricted to role: ADMIN
/add-learner	Register New Learner form, restricted to role: ADMIN
/csv-upload	CSV bulk learner import panel, restricted to role: ADMIN
/mentor	Mentor Dashboard, restricted to role: MENTOR
/assessment	Assessment log submission form, restricted to role: MENTOR
/prediction	AI Placement prediction console, restricted to role: ADMIN
/analytics	Visual charting and stats dashboard, restricted to roles: ADMIN, MENTOR
/notifications	Notification history feed, restricted to roles: MENTOR, LEARNER